

Multi-Tenant SaaS — Functional Document

Overview

This is a production-ready **multi-tenant SaaS project management platform** built with Laravel 11. Each tenant (company) is completely isolated in its own MySQL database. The system supports team collaboration with projects, tasks, comments, attachments, real-time updates, and a full subscription billing cycle powered by Stripe.

Core Capabilities

Capability	Description
Multi-tenancy	Per-database isolation via <code>stancyl/tenancy</code> v3.10
Authentication	Laravel Sanctum — token-based API + session-based Web
Authorization	Role-based access control via Spatie Permission + Policies
Billing	Stripe Checkout — 3 plans (Starter / Professional / Enterprise), monthly & yearly cycles
Project Management	Projects → Tasks → Comments / Attachments, status transitions, history
Real-time	Laravel Reverb (WebSocket broadcasting) — live task and subscription events

Tech Stack

- **Backend:** Laravel 11, PHP 8.2
- **Database:** MySQL (central DB + per-tenant DB), Redis (cache + queues)
- **Auth:** Sanctum, Spatie Permission
- **Billing:** Stripe SDK, DomPDF (invoice generation)
- **Real-time:** Laravel Reverb (self-hosted), Pusher (fallback)
- **Frontend:** Blade templates, Tailwind CSS, Vite
- **Architecture:** Domain-Driven Design (DDD) — `app/Domain/{Feature}/`

3. Project & Task Management Flow

```

Authenticated user on tenant subdomain (e.g., company.app.test)
|
├─ Projects
|   ├─ Create → ProjectService → ProjectCreated event → ActivityLog
|   ├─ Update → ProjectService → ProjectUpdated / ProjectStatusChanged events
|   └─ Delete (CompanyAdmin only) → soft delete, restorable
|
└─ Tasks (within a Project)
    ├─ Create → TaskService → TaskCreated event
    |   └─ If assigned → TaskAssigned event → Email + WebSocket push
    |       └─ History entry written
    |
    ├─ Update status → TaskStatusChanged event
    |   ├─ State machine: todo → in_progress → in_review → blocked → done/cancelled
    |   ├─ HandleTaskStatusChanged → sets completed_at if done
    |   └─ WebSocket broadcast → client updates live
    |
    ├─ Comments → TaskCommentAdded event → Email + WebSocket push
    └─ Attachments → MIME whitelist enforced, 20 MB cap, tenant-scoped storage
  
```

4. Real-time Broadcast Flow

```

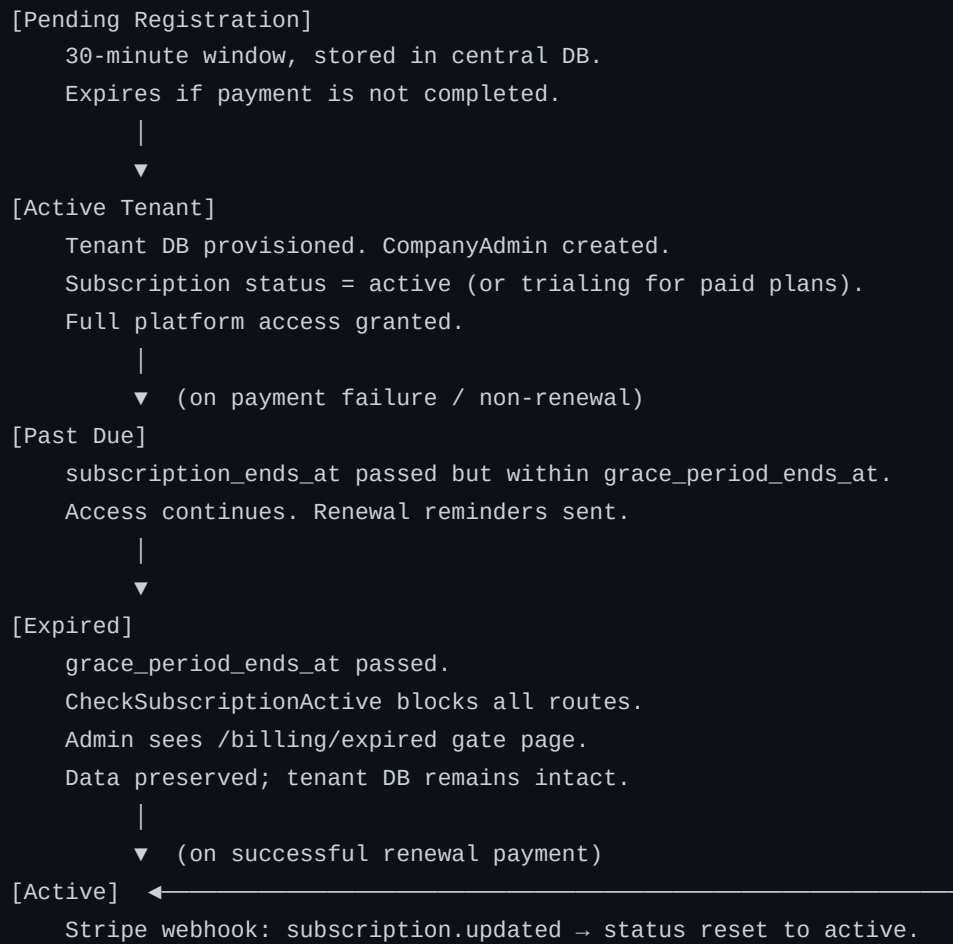
Domain Event fires (e.g., TaskAssigned, TaskCommentAdded, ProjectUpdated)
|
├─ Queued Broadcast Listener (afterCommit = true)
|   └─ Broadcasts on tenant-scoped private channel
|       |
|       Channel topology:
|       ├─ private-tenant.{tenantId} ← subscription events
|       ├─ private-tenant.{tenantId}.project.{projectId} ← project mutations
|       ├─ private-tenant.{tenantId}.task.{taskId} ← comment threads
|       └─ private-tenant.{tenantId}.user.{userId} ← task assignments
|       |
|       └─ Laravel Reverb (WebSocket server)
|           └─ Pushes to subscribed browser clients → live UI update
  
```

5. Billing & Subscription Management Flow

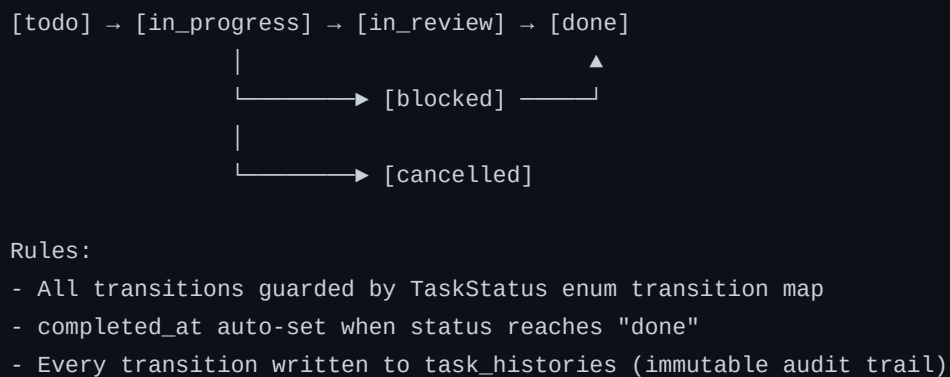
```
Tenant operational
|
|─ Stripe webhook: invoice.payment_succeeded
|   └─ Creates Invoice record in tenant DB
|       └─ InvoicePaidNotification → receipt email to admin
|
|─ Stripe webhook: customer.subscription.updated/deleted
|   └─ Updates SubscriptionStatus on Tenant model
|
|─ Daily: billing:check-expired (console command via Scheduler)
|   └─ Finds subscriptions past subscription_ends_at
|       └─ If still in grace period → status = past_due
|           └─ If past grace period → status = expired
|               └─ SubscriptionExpired event
|                   └─ SubscriptionExpiredNotification → email to admin
|                       └─ SubscriptionExpiredBroadcast → live banner on tenant UI
|
|─ CheckSubscriptionActive middleware
|   └─ Gates all tenant routes
|       └─ Returns 402 JSON (API) or redirects to /subscription/expired (Web)
```

Lifecycle

Tenant Lifecycle



Task Status Lifecycle



Project Status Lifecycle

```
[planning] → [active] → [on_hold] → [completed]
                                   → [cancelled]
```

- Status changes fire `ProjectStatusChanged` event
- `HandleProjectStatusChanged` writes a dedicated activity log entry

Subscription Plan Lifecycle

```
[Starter – Free]    No Stripe; provisioned directly.
[Professional]      $29/month or $290/year → Stripe Checkout
[Enterprise]        $99/month or $990/year → Stripe Checkout
```

Plan pricing seeded from `PlanSeeder`.

Stripe Price IDs injected via `.env` (per billing cycle).

Process

Tenant Provisioning Process

1. `InitiateRegistrationAction` validates registration data and creates a `PendingRegistration` record in the central DB.
2. For paid plans, a Stripe Checkout session is created and the user is redirected to Stripe.
3. On payment success, Stripe posts `checkout.session.completed` to `POST /api/stripe/webhook`.
4. `HandleStripeWebhookAction` retrieves the pending registration and calls `ProvisionTenantFromPendingAction`.
5. `ProvisionTenantFromPendingAction`:
 - Creates the `Tenant` record in the central DB with Stripe customer ID and subscription fields.
 - Runs `CreateDatabase` + `MigrateDatabase` jobs to provision the tenant's isolated MySQL database.
 - Creates the `CompanyAdmin` user in the tenant DB and assigns the `CompanyAdmin` role.
 - Marks `PendingRegistration` as `completed`.
 - Dispatches `SubscriptionActivated` event → welcome email notification.

Request Lifecycle (Tenant-Scoped Request)

1. HTTP request arrives on tenant subdomain (e.g., `acme.app.test`).

2. `stanc/tenancy` `InitializeTenancyByDomain` middleware identifies the tenant and switches the DB connection to the tenant's isolated database.
3. `CheckSubscriptionActive` middleware verifies the subscription status; blocks with 402 or redirect if expired.
4. `auth:sanctum` middleware verifies the bearer token or session.
5. `EnforceTenantScope` middleware attaches a global Eloquent scope to prevent cross-tenant data leakage.
6. Request reaches the controller → FormRequest validation → Action/Service → Repository → Response.

Cross-Tenant Security Process

Three independent layers prevent data bleed between tenants:

Layer	Mechanism
Database	<code>stanc/tenancy</code> switches connection; each tenant has its own MySQL DB
Eloquent	Global scope <code>EnforceTenantScope</code> appended to all tenant models
Policy	All <code>Policy</code> classes verify the authenticated user belongs to the same tenant context

Event & Broadcast Process

1. A domain action (e.g., `TaskService::assign`) dispatches a domain event (`TaskAssigned`).
2. `EventServiceProvider` maps the event to both a domain listener and a broadcast listener.
3. The broadcast listener is queued (`ShouldQueue`), runs on the `broadcasts` queue, fires `afterCommit = true` to avoid stale reads.
4. The broadcast listener pushes a `Broadcast` event class onto the appropriate private channel via Laravel Reverb.
5. The frontend JS client (subscribed via Pusher JS SDK) receives the event and updates the UI without a page reload.

Invoice & PDF Process

1. Stripe fires `invoice.payment_succeeded` webhook.
2. `HandleStripeWebhookAction` creates an `Invoice` record in the tenant DB with amount, currency, Stripe invoice ID, and `paid_at` timestamp.
3. Admin visits `/billing` (tenant-scoped) to see invoice history.
4. `GET /billing/invoices/{id}/download` triggers `InvoiceService::generate()` which renders `billing/invoice-pdf.blade.php` via DomPDF and streams the PDF to the browser.

Subscription Expiry Process

1. `billing:check-expired` console command runs daily via Laravel Scheduler.

2. Queries tenants where `subscription_ends_at < now()` and `subscription_status != expired` .
3. If `grace_period_ends_at > now()` → sets `subscription_status = past_due` .
4. If `grace_period_ends_at < now()` → sets `subscription_status = expired` , fires `SubscriptionExpired` event.
5. `SubscriptionExpired` dispatches:
 - `SubscriptionExpiredNotification` → email to CompanyAdmin.
 - `SubscriptionExpiredBroadcast` → WebSocket push showing an expiry banner on the live session.
6. On the next request, `CheckSubscriptionActive` middleware intercepts and redirects to `/subscription/expired` .

Last updated: 2026-05-15